# 8086 Assembly Practice Problems

*Loops, Bit Manipulation, and String Processing (Problems 1–7)*

## Problem 1

***Write an 8086 assembly program that counts the number of 1-bits (set bits) in a 16-bit word stored in memory.***

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM DW 1234h  ; Sample data 0001 0010 0011 0100
    COUNT DB 0     ; Bit counter
.CODE
MAIN PROC
    mov ax, @data
    mov ds, ax
    MOV CX, 16     ; 16 bits in word
    MOV AX, NUM
LOOP_START:
    RCL AX, 1      ; Rotate left through carry
    JNC NO_INC     ; If CF=0, skip increment
    INC COUNT
NO_INC:
    LOOP LOOP_START
    ; COUNT now has number of 1-bits
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Problem 2

***Write an 8086 assembly program that calculates the sum of the first N natural numbers (from 1 to N).***

```
.MODEL SMALL
.STACK 100H
.DATA
    N DW 10            ; Number for sum (first N naturals)
    SUM DW 0           ; To store the result
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
```

```
    MOV CX, N           ; Load N into CX
    MOV AX, 0           ; Initialize sum to 0

SUM_LOOP:
    ADD AX, CX          ; Add current CX to AX
    LOOP SUM_LOOP       ; Decrement CX, loop if CX != 0

    MOV SUM, AX         ; Store the result in memory

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Problem 3

***Write an 8086 assembly program that checks whether a given number N is prime or not prime.***

```
.MODEL SMALL
.STACK 100H
.DATA
    N DW 14             ; Number to check (e.g., 13 is prime, 15 is not)
    IS_PRIME DB 0       ; 1 if prime, 0 if not
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, N           ; Load N
    CMP AX, 2           ; If N < 2, not prime
    JB NOT_PRIME_LABEL

    MOV CX, AX          ; CX = N
    SUB CX, 2           ; Number of iterations: from 2 to N-1 (N-2 times)
    CMP CX, 0
    JE SET_PRIME        ; If CX=0 (N=2), prime

    MOV BX, 2           ; Start divisor at 2

PRIME_LOOP:
    MOV DX, 0           ; Clear DX for division
    MOV AX, N           ; Reload N into AX
    DIV BX              ; AX / BX, remainder in DX
    CMP DX, 0           ; If remainder 0, divisible
    JE NOT_PRIME_LABEL  ; Jump to not prime

    INC BX              ; Next divisor
    LOOP PRIME_LOOP     ; Repeat

SET_PRIME:
```

```
    MOV IS_PRIME, 1    ; Falls through: prime
    JMP EXIT

NOT_PRIME_LABEL:
    MOV IS_PRIME, 0    ; Not prime

EXIT:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Problem 4

***Write an 8086 assembly program to compute the factorial of a given non-negative integer NUM (stored as a byte in memory). The program should calculate NUM!***

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM DB 5           ; Change this to the number for factorial (e.g., 5! = 120)
    RESULT DW 0        ; To store the result
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, NUM        ; Load number into AL
    MOV AH, 0          ; Clear AH for word operations
    MOV CX, AX         ; Set loop counter to n
    MOV AX, 1          ; Start with 1 (0! and 1! are 1)

    CMP CX, 0          ; If n=0, result=1
    JE STORE_RESULT

FACTORIAL_LOOP:
    MUL CX             ; AX = AX * CX
    LOOP FACTORIAL_LOOP; Decrement CX, loop if CX != 0

STORE_RESULT:
    MOV RESULT, AX     ; Store the result

    ; Optional: Display result (convert to ASCII and output)
    ; For simplicity, assume we inspect in emulator

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Problem 6

***Write an 8086 assembly program to reverse the bit order of an 8-bit binary number.***
***Input: 10100011***
***Output: 11000101 (reverse of input 10100011)***

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV DL, 10100011b   ; 0xA3
    MOV CL, 8           ; 8 bits
    XOR BL, BL          ; result in BL (lower byte of BX)

REV_LOOP:
    ROR DL, 1           ; LSB of DL -> CF, bits shift right
    RCL BL, 1           ; CF -> LSB of BL, BL shifts left
    DEC CL
    JNZ REV_LOOP

    ; Now BL = 11000101b = 0xC5
    ; BX = 00C5h

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Problem 7

***Write an 8086 assembly program that takes a $-terminated string stored in memory, converts all lowercase letters (a–z) in the string to uppercase, and leaves all other characters unchanged. After conversion, display the message "Capitalized: " followed by the modified string, then exit to DOS.***

```
.MODEL SMALL
.STACK 100H

.DATA
    STR DB "heLlo world! 8086 asm$"
    MSG DB "Capitalized: $"

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
```

```
    MOV ES, AX

    CLD

    MOV SI, OFFSET STR
    MOV DI, OFFSET STR

CAP_LOOP:
    LODSB

    CMP AL, '$'
    JE  DONE_STORE_END

    CMP AL, 'a'
    JB  STORE_CHAR
    CMP AL, 'z'
    JA  STORE_CHAR
    SUB AL, 20H

STORE_CHAR:
    STOSB
    JMP CAP_LOOP

DONE_STORE_END:
    STOSB

    MOV DX, OFFSET MSG
    MOV AH, 9
    INT 21H

    MOV DX, OFFSET STR
    MOV AH, 9
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```